TASK-9

GREEDY TECHNIQUE - DIJKSTRA'S ALGORITHM

AIM: To find the shortest path from a source node to all other nodes in a weighted graph.

ALGORITHM:
1. Intialize
   - → Distance arrays dist[] = ∞ for all nodes
   - → set source distance : dist[src]=0
   - → Use a min heap (priority) to store(distance, node)
2. Push (0, src) into the heap
3. while heap isn't empty :
   - * pop the node with the smallest distance
   - * If already visited → skip
   - * Mark it as visited
   - * For each neighbour :
     - → If dist [current] + weight < dist[neighbour]:
     - → Update dist [neighbour]
     - → push updated value into heap

PROGRAM:

```
#include <assert.h>
#include <etype.h>
#include <limits.h>
#include <math.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
char* readline();
char* trim(char*);
char* rtrim(char*);
```

```
    f printf(fptr, "\n");
  }
    fclose(fptr);
    return 0;
}
char* readline(){
        size_t alloc_length = 1024;
        size_t data_length = 0;
    char* data = malloc(alloc_length);
      while (true){
          char* cursor = data + data_length;
        char* line = fgets(cursor, alloc_length - data_length,
                                                    stdin);
         if(! line){
            break;
      }
    data_length += strlen(cursor);
if (data_length < alloc_length - 1 || data[data_length - 1] ==
                                                    "\n"){
          break;
      }
      alloc_length <<= 1;
    data = realloc(data, alloc_length);
          if (! data){
          data = "\0";
            break;
        }
      }
  if (data[data_length - 1] = "\n"){
        data[data_length - 1] = "\0";
      data = realloc(data, data_length);
          if (! data){
            data = "\0";
        }
    }else{
```

```
    data = realloc (data, data_length + 1);
        if (! data) {
            data = "\0";
        } else {
            data [data_length] = "\0";
        }
    }
    return data;
}
char* ltrim (char* str){
        if (! str){
            return '\0';
        }
        if (! * str){
            return str;
        }
        while (* str ! = "\0" && isspace (*str)){
            str ++;
        }
        return str;
    }
    char* rtrim (char* str){
        if (! str){
            return "\0";
        }
        if (! * str){
            return str;
        }
        char* end = Str + Strlen (str) - 1;
        while (end >= str && isspace (* end)){
            end - -;
        }
        *(end + 1) = "\0";
        return str;
    }
```

```
Char ** split_string (Char* str){
    char** splits = NULL;
    Char* token = strtok(str," ");
        int spaces = 0;
    while (token){
     splits = realloc (splits, size of (char *)* ++spaces);
    if (! splits){
        return splits;
    }
    splits [spaces-1] = token;
     token = strtok (NULL," ");
    }
    return splits;
}
int parse - int (char* str){
    Char* endptr;
    int value = strtol (str, &end ptr ,10);
    if (end ptr == str //* end ptr != "\0"){
        exit (EXIT_FAILURE);
    }
    return value;
}
```

VEL TECH	
No.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	

RESULT: Thus the above task has been executed & verified sucessfully.